

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Applied Business Analytics Practical**

**Practical – V**

|                                |                                     |
|--------------------------------|-------------------------------------|
| Roll No.: T005                 | Name: Ritesh Chaubey                |
| Class:TYIT                     | Batch:1                             |
| Date of Assignment: 21/08/2026 | Date/Time of Submission: 21/08/2026 |

**AIM: Probability and Probability Distributions.**

**A)Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.**

1. Simulate 1,000 dice rolls and display the first 20 outcomes.
2. Calculate the experimental probability of getting each number from 1 to 6.
3. Calculate the theoretical probability of getting each number from 1 to 6.
4. Find the probability of getting 1, 2, 3, 4, 5, 6.
5. Find the probability of getting an even number.
6. Find the probability of getting an odd number.
7. Find the probability of getting a number greater than 4.
8. Find the probability of getting a number less than 4.
9. Find the probability of getting a number greater than or equal to 4.
10. Create a bar chart showing the frequency of outcomes 1 to 6.
11. Create a bar chart showing the experimental probability of outcomes 1 to 6.
12. Create a pie chart showing the percentage distribution of dice outcomes.
13. Create a line chart comparing experimental probability and theoretical probability for outcomes 1 to 6.
14. Create a chart showing the frequency of Even vs Odd outcomes.
15. Create a chart showing the experimental probability of:
  - Even numbers
  - Odd numbers
  - Numbers greater than 4
  - Numbers less than 4

**CODE:**

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
# 1. Simulate 1000 dice rolls
dice_rolls = np.random.randint(1, 7, 1000)
# Display first 20 outcomes
```

```
print("First 20 Outcomes:")
print(dice_rolls[:20])
# 2. Experimental Probability
frequency = np.bincount(dice_rolls)[1:]
experimental_probability = frequency / 1000
# 3. Theoretical Probability
theoretical_probability = np.repeat(1/6, 6)
print("\nOutcome | Frequency | Experimental | Theoretical")
for i in range(6):
    print(f'{i+1} | {frequency[i]} | {experimental_probability[i]:.3f} | {theoretical_probability[i]:.3f}')
# 4. Probability of getting 1 to 6
print("\nProbability of Each Number")
for i in range(6):
    print(f'P({i+1}) = {experimental_probability[i]:.3f}')
# 5-9 Required Probabilities
print("\nProbability of Even =", np.mean(dice_rolls % 2 == 0))
print("Probability of Odd =", np.mean(dice_rolls % 2 == 1))
print("Probability Greater than 4 =", np.mean(dice_rolls > 4))
print("Probability Less than 4 =", np.mean(dice_rolls < 4))
print("Probability Greater than or Equal to 4 =", np.mean(dice_rolls >= 4))
# 10. Frequency Bar Chart
plt.figure(figsize=(6,4))
plt.bar(range(1,7), frequency)
plt.title("Frequency of Dice Outcomes")
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.show()
# 11. Experimental Probability Bar Chart
plt.figure(figsize=(6,4))
plt.bar(range(1,7), experimental_probability)
plt.title("Experimental Probability of Dice Outcomes")
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.show()
# 12. Pie Chart
plt.figure(figsize=(5,5))
plt.pie(frequency, labels=[1,2,3,4,5,6], autopct="%1.1f%%")
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()
# 13. Line Chart
plt.figure(figsize=(6,4))
plt.plot(range(1,7), experimental_probability, marker="o", label="Experimental")
plt.plot(range(1,7), theoretical_probability, marker="s", label="Theoretical")
plt.legend()
plt.title("Experimental vs Theoretical Probability")
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.show()
# 14. Even vs Odd Frequency
even_count = np.sum(dice_rolls % 2 == 0)
```

```

odd_count = np.sum(dice_rolls % 2 == 1)
plt.figure(figsize=(5,4))
plt.bar(["Even", "Odd"], [even_count, odd_count])
plt.title("Even vs Odd Frequency")
plt.show()
# 15. Experimental Probability Chart
categories = ["Even", "Odd", ">4", "<4"]
values = [
    np.mean(dice_rolls % 2 == 0),
    np.mean(dice_rolls % 2 == 1),
    np.mean(dice_rolls > 4),
    np.mean(dice_rolls < 4)
]
plt.figure(figsize=(6,4))
plt.bar(categories, values)
plt.title("Experimental Probabilities")
plt.ylabel("Probability")
plt.show()

```

**OUTPUT:**

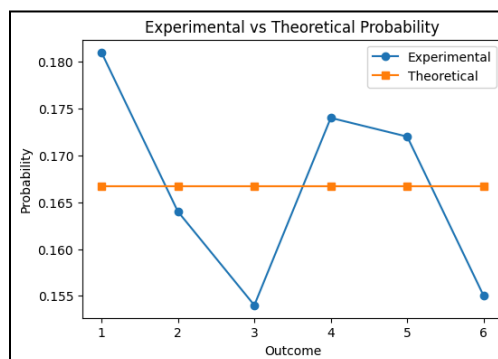
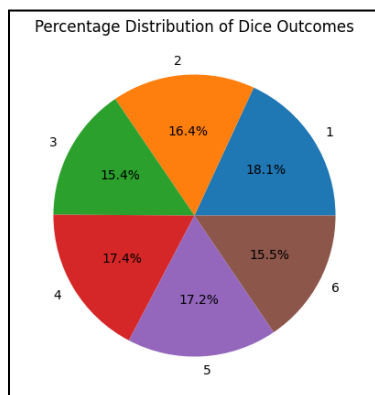
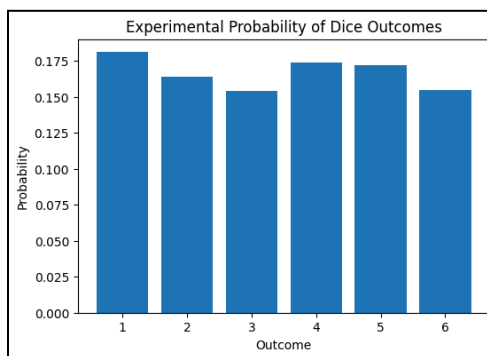
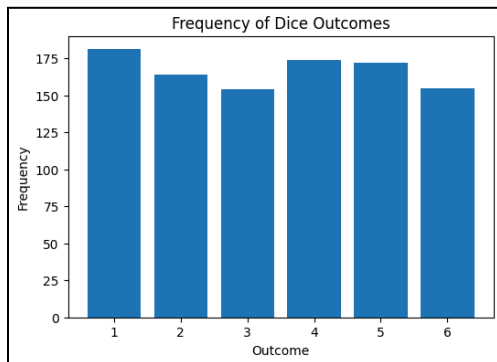
First 20 Outcomes:

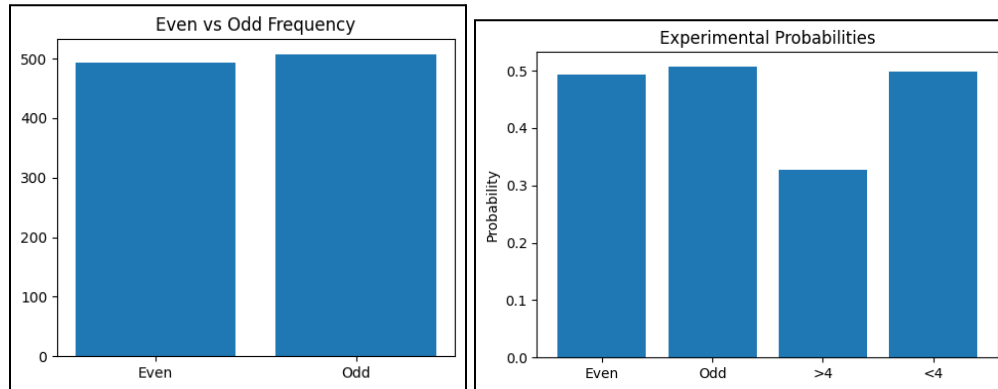
[4 5 3 5 5 2 3 3 3 5 4 3 6 5 2 4 6 6 2 4]

Probability of Even = 0.493  
 Probability of Odd = 0.507  
 Probability Greater than 4 = 0.327  
 Probability Less than 4 = 0.499  
 Probability Greater than or Equal to 4 = 0.501

| Outcome | Frequency | Experimental | Theoretical |
|---------|-----------|--------------|-------------|
| 1       | 181       | 0.181        | 0.167       |
| 2       | 164       | 0.164        | 0.167       |
| 3       | 154       | 0.154        | 0.167       |
| 4       | 174       | 0.174        | 0.167       |
| 5       | 172       | 0.172        | 0.167       |
| 6       | 155       | 0.155        | 0.167       |

Probability of Each Number  
 P(1) = 0.181  
 P(2) = 0.164  
 P(3) = 0.154  
 P(4) = 0.174  
 P(5) = 0.172  
 P(6) = 0.155





**B) Generate random numbers from various probability distributions (Normal, Uniform, Exponential) and analyze their properties.**

1. Generate 1,000 random numbers from a Normal Distribution with:
  - Mean = 50
  - Standard Deviation = 10
2. Calculate the mean, median, mode, variance, and standard deviation of the generated data.
3. Calculate the following statistics for the generated Normal distribution:
  - Mean
  - Median
  - Variance
  - Standard Deviation
  - Minimum
  - Maximum
4. Create a histogram of the Normal distribution and analyze its shape.
5. Create a box plot of the Normal distribution.
6. Calculate the percentage of values that fall between  $40 \leq X \leq 60$ .
7. Calculate the percentage of values that fall between  $30 \leq X \leq 70$ .
8. Generate 1,000 random numbers from a Uniform Distribution between 0 and 100.
9. Create a histogram of the Uniform distribution.
10. Create a box plot of the Uniform distribution.
11. Generate 1,000 random numbers from an Exponential Distribution with scale = 2.
12. Create a histogram of the Exponential distribution.
13. Create a box plot and identify possible extreme values.
14. Calculate the percentage of values greater than 2.
15. Calculate the percentage of values greater than 5.

**CODE:**

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
```

```
np.random.seed(42)
# 1. Normal Distribution
normal_data = np.random.normal(50, 10, 1000)
# 2 & 3. Statistics
mean = np.mean(normal_data)
median = np.median(normal_data)
mode = Counter(np.round(normal_data)).most_common(1)[0][0]
variance = np.var(normal_data)
std = np.std(normal_data)
print("Normal Distribution Statistics")
print("Mean =", mean)
print("Median =", median)
print("Mode =", mode)
print("Variance =", variance)
print("Standard Deviation =", std)
print("Minimum =", np.min(normal_data))
print("Maximum =", np.max(normal_data))
# 4. Histogram
plt.figure(figsize=(6,4))
plt.hist(normal_data, bins=20)
plt.title("Normal Distribution Histogram")
plt.show()
# 5. Box Plot
plt.figure(figsize=(6,2))
plt.boxplot(normal_data, vert=False)
plt.title("Normal Distribution Box Plot")
plt.show()
# 6 & 7. Percentages
print("40 <= X <= 60 =", np.mean((normal_data>=40)&(normal_data<=60))*100,"%")
print("30 <= X <= 70 =", np.mean((normal_data>=30)&(normal_data<=70))*100,"%")
# 8. Uniform Distribution
uniform_data = np.random.uniform(0,100,1000)
# 9. Histogram
plt.figure(figsize=(6,4))
plt.hist(uniform_data, bins=20)
plt.title("Uniform Distribution Histogram")
plt.show()
# 10. Box Plot
plt.figure(figsize=(6,2))
plt.boxplot(uniform_data, vert=False)
plt.title("Uniform Distribution Box Plot")
plt.show()
# 11. Exponential Distribution
exponential_data = np.random.exponential(scale=2, size=1000)
# 12. Histogram
plt.figure(figsize=(6,4))
plt.hist(exponential_data, bins=20)
plt.title("Exponential Distribution Histogram")
plt.show()
# 13. Box Plot
plt.figure(figsize=(6,2))
plt.boxplot(exponential_data, vert=False)
plt.title("Exponential Distribution Box Plot")
plt.show()
# 14 & 15. Percentages
print("Percentage Greater Than 2 =", np.mean(exponential_data>2)*100,"%")
print("Percentage Greater Than 5 =", np.mean(exponential_data>5)*100,"%")
```

**OUTPUT:**

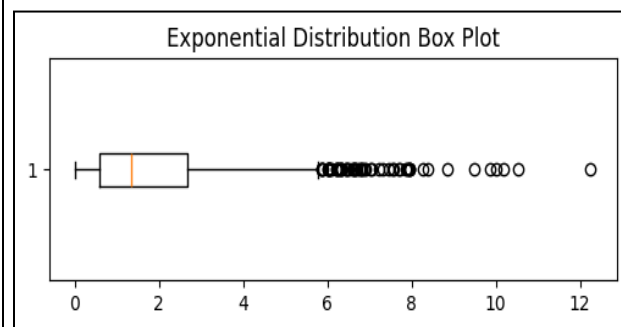
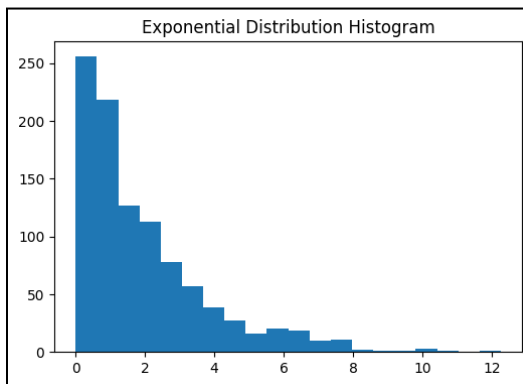
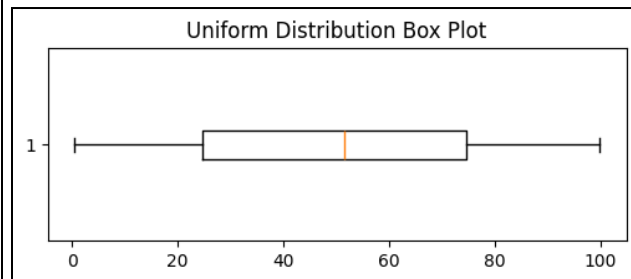
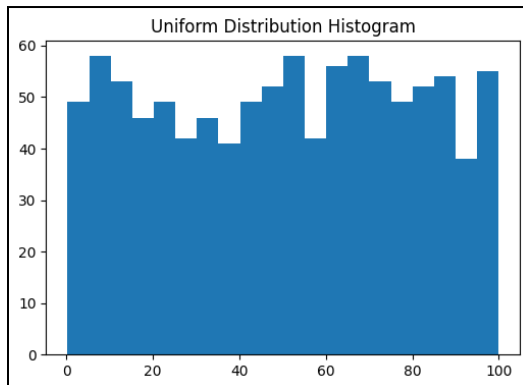
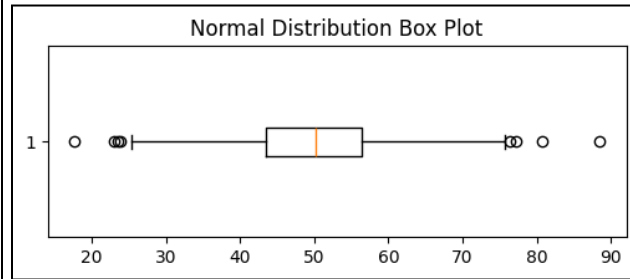
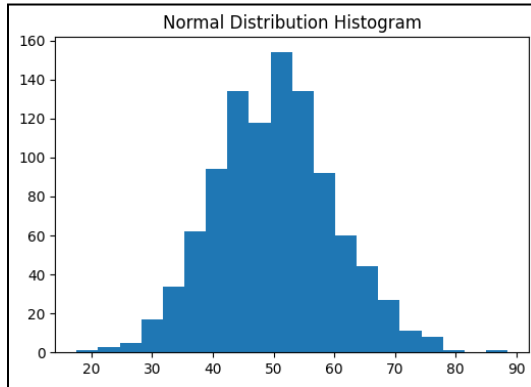
```

*** Normal Distribution Statistics
Mean = 50.193320558223256
Median = 50.25300612234888
Mode = 52.0
Variance = 95.79049897315173
Standard Deviation = 9.787262077473542
Minimum = 17.58732659930927
Maximum = 88.52731490654722

```

$40 \leq X \leq 60 = 69.8 \%$

$30 \leq X \leq 70 = 95.89999999999999 \%$



Percentage Greater Than 2 = 36.5 %

Percentage Greater Than 5 = 8.200000000000001 %

**C) Generate random numbers from various probability distributions (Normal, Uniform, Exponential) and analyze their properties.**

1. **Simulate 1,000 coin tosses and calculate the experimental probability of:**
  - Getting Heads
  - Getting Tails
2. **Calculate the theoretical probability of Heads and Tails.**
3. **Create a bar chart showing the frequency of Heads and Tails.**
4. **Create a pie chart showing the percentage of Heads and Tails.**
5. **If a coin is tossed 10 times and Heads occurs 8 times, calculate the experimental probability of Heads.**
6. **Simulate 10,000 coin tosses and create a complete analysis containing:**
  - Total tosses
  - Number of Heads
  - Number of Tails
  - Experimental probability of Heads
  - Experimental probability of Tails
  - Theoretical probabilities
  - Difference between experimental and theoretical probabilities
  - Bar chart
  - Pie chart

**CODE:**

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
# 1. Simulate 1000 Coin Tosses
coin = np.random.choice(["Heads", "Tails"], 1000)
heads = np.sum(coin=="Heads")
tails = np.sum(coin=="Tails")
print("1000 Tosses")
print("Heads =", heads)
print("Tails =", tails)
print("Experimental Probability of Heads =", heads/1000)
print("Experimental Probability of Tails =", tails/1000)
# 2. Theoretical Probability
print("\nTheoretical Probability")
print("Heads = 0.5")
print("Tails = 0.5")
# 3. Bar Chart
plt.figure(figsize=(5,4))
plt.bar(["Heads", "Tails"], [heads, tails])
plt.title("Heads vs Tails Frequency")
plt.show()
# 4. Pie Chart
plt.figure(figsize=(5,5))
plt.pie([heads, tails], labels=["Heads", "Tails"], autopct="%1.1f%%")
plt.title("Heads vs Tails Percentage")
plt.show()
# 5. Heads occurs 8 times in 10 tosses
```

```

print("\nExperimental Probability of Heads (8 out of 10) =", 8/10)
# 6. Simulate 10000 Tosses
coin_large = np.random.choice(["Heads","Tails"],10000)
heads_large = np.sum(coin_large=="Heads")
tails_large = np.sum(coin_large=="Tails")
exp_heads = heads_large/10000
exp_tails = tails_large/10000
print("\n10000 Tosses Analysis")
print("Total Tosses =",10000)
print("Heads =",heads_large)
print("Tails =",tails_large)
print("Experimental Probability of Heads =",exp_heads)
print("Experimental Probability of Tails =",exp_tails)
print("Theoretical Probability = 0.5")
print("Difference (Heads) =",abs(exp_heads-0.5))
print("Difference (Tails) =",abs(exp_tails-0.5))
# Bar Chart
plt.figure(figsize=(5,4))
plt.bar(["Heads","Tails"], [heads_large,tails_large])
plt.title("10000 Tosses Frequency")
plt.show()
# Pie Chart
plt.figure(figsize=(5,5))
plt.pie([heads_large,tails_large], labels=["Heads","Tails"], autopct="%1.1f%%")
plt.title("10000 Tosses Percentage")
plt.show()

```

**OUTPUT:**

```

*** 1000 Tosses
Heads = 490
Tails = 510
Experimental Probability of Heads = 0.49
Experimental Probability of Tails = 0.51

Theoretical Probability
Heads = 0.5
Tails = 0.5

```

Experimental Probability of Heads (8 out of 10) = 0.8

```

10000 Tosses Analysis
Total Tosses = 10000
Heads = 5012
Tails = 4988
Experimental Probability of Heads = 0.5012
Experimental Probability of Tails = 0.4988
Theoretical Probability = 0.5
Difference (Heads) = 0.0011999999999999789
Difference (Tails) = 0.0011999999999999789

```

